UNIDAD 7

Evaluación, métricas, validación y generalización

Cómo saber si un modelo realmente aprendió

Guía de estudio: Programador Universitario de Sistemas

CONTENIDOS

1. Introducción — ¿Cómo sabemos si un modelo es bueno?
2. Parte I — Evaluación de modelos de clasificación
3. Parte II — Evaluación de modelos de regresión
4. Parte III — Train, validation, test y validación cruzada
5. Parte IV — Generalización, underfitting y overfitting
6. Parte V — Aplicación práctica y cierre

Introducción

¿Cómo sabemos si un modelo es bueno?

En la unidad anterior comenzamos a trabajar con modelos concretos de aprendizaje supervisado. Vimos que una misma situación puede ser abordada mediante diferentes estrategias y que cada modelo tiene una forma particular de aprender y de producir una predicción. Sin embargo, quedó planteada una pregunta mucho más importante que simplemente saber entrenar un modelo:

> ***¿Cómo podemos determinar si ese modelo realmente es bueno?***

Supongamos que construimos un modelo capaz de predecir si un cliente comprará un producto y, después de entrenarlo, obtenemos un **95 % de aciertos**. A primera vista podríamos concluir que tenemos un modelo excelente. Pero inmediatamente aparece una pregunta: ¿95 % de aciertos sobre qué datos? Si evaluamos el modelo utilizando exactamente los mismos datos que utilizó para aprender, ese porcentaje puede resultar engañoso.

Un modelo de Machine Learning no tiene como objetivo demostrar que puede reproducir los datos que ya conoce. Su verdadera utilidad aparece cuando recibe una observación que no estuvo presente durante el aprendizaje y consigue producir una predicción razonable. Por eso, a partir de esta clase, debemos comenzar a pensar en términos de generalización.

Parte I — Evaluación de clasificación

7.1 — Predicción, valor real y error

Todo proceso de evaluación comienza comparando lo que el modelo predijo con lo que realmente ocurrió. Supongamos que nuestro sistema analiza un cliente y determina que realizará una compra. Posteriormente sabemos que efectivamente compró: la predicción fue correcta. En otro caso, el modelo puede predecir que el cliente comprará, pero finalmente no realiza ninguna compra: la predicción fue incorrecta. La diferencia entre ambos constituye el punto de partida para analizar el error del modelo.

En regresión, el error puede tener una magnitud concreta: si el modelo predice que un cliente gastará $200.000 y finalmente gasta $180.000, existe una diferencia de $20.000. En clasificación, en cambio, normalmente comenzamos preguntando si la predicción pertenece a la clase correcta — aunque, como veremos, **no todos los errores tienen la misma importancia**.

> *Un sistema que detecta fraude puede equivocarse de dos maneras muy distintas: marcar una transacción legítima como fraude, o dejar pasar una transacción fraudulenta. Ambos son errores, pero sus consecuencias no son iguales. Por eso no alcanza con contar únicamente cuántas predicciones fueron correctas.*

7.2 — Función de pérdida y desempeño

Durante el entrenamiento, los modelos necesitan algún mecanismo que permita determinar qué tan buenas o malas son sus predicciones. Para ello aparecen las funciones de pérdida o funciones de costo: una predicción que se aleja mucho del resultado esperado debería producir una penalización mayor que una predicción cercana.

Es importante diferenciar dos ideas que muchas veces se confunden: la **función de pérdida** se utiliza dentro del proceso de aprendizaje para orientar el entrenamiento, mientras que las **métricas** nos ayudan a analizar el desempeño del modelo una vez entrenado. No necesariamente debemos utilizar la misma medida para entrenar y para comunicar o interpretar el desempeño final.

7.3 — La matriz de confusión

Para comprender correctamente la evaluación de un clasificador necesitamos abandonar momentáneamente la idea de "acertó o falló" y observar qué tipo de acierto o error produjo. Supongamos que queremos detectar si una operación es fraudulenta. A partir de comparar predicción y realidad aparecen cuatro posibilidades fundamentales:

- **Verdadero positivo (TP):** el modelo predice positivo y el caso efectivamente es positivo.
- **Verdadero negativo (TN):** el modelo predice negativo y el caso efectivamente es negativo.
- **Falso positivo (FP):** el modelo predice positivo pero el caso realmente era negativo.
- **Falso negativo (FN):** el modelo predice negativo pero el caso realmente era positivo.

MATRIZ DE CONFUSIÓN — ESQUEMA

Predicción ↓ / Real →	**Real positivo**	**Real negativo**
Predicción positiva	**TP**	**FP**
Predicción negativa	**FN**	**TN**

Su importancia es enorme porque nos permite dejar de observar solamente el porcentaje global de aciertos y comenzar a estudiar la naturaleza de los errores.

7.4 — Un ejemplo sencillo

Supongamos que tenemos 100 operaciones y nuestro modelo produjo los siguientes resultados:

MATRIZ DE CONFUSIÓN — EJEMPLO CON 100 OPERACIONES

Predijo ↓ / Real →	**Real fraude**	**Real legítima**
Predijo fraude	**18**	**7**
Predijo legítima	**5**	**70**

Tenemos entonces **TP=18**, **FP=7**, **FN=5** y **TN=70**. El modelo acertó 18 fraudes y 70 operaciones legítimas: 88 predicciones correctas sobre 100.

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Accuracy = (18 + 70) / 100 = 0.88   →   88 %
```

Hasta aquí podríamos decir que el modelo tiene un 88 % de accuracy. Pero todavía no sabemos si ese resultado es suficientemente bueno para nuestro problema.

7.5 — Accuracy

La accuracy representa la proporción de predicciones correctas respecto del total de observaciones. Es una métrica intuitiva y útil en muchos problemas, pero tiene una limitación importante: puede ocultar problemas cuando las clases están desbalanceadas.

Imaginemos un sistema de detección de fraude donde de 10.000 operaciones solamente 100 son fraudulentas. Si un modelo predice que todas las operaciones son legítimas, acertaría 9.900 de las 10.000:

```
Accuracy = 9900 / 10000 = 0.99   →   99 %
```

> *A primera vista parece extraordinario. Pero el modelo no detectó ningún fraude: para un sistema cuyo objetivo principal es detectar operaciones fraudulentas, ese modelo sería prácticamente inútil. Esto demuestra por qué una métrica aparentemente excelente puede esconder un comportamiento inadecuado.*

7.6 — Precision

La precision responde a una pregunta diferente:

> *De todos los casos que el modelo clasificó como positivos, ¿cuántos realmente eran positivos?*

```
Precision = TP / (TP + FP)
Precision = 18 / (18 + 7) = 0.72   →   72 %
```

De todas las operaciones que el modelo marcó como fraudulentas, aproximadamente el 72 % realmente eran fraudulentas. Esta métrica resulta especialmente importante cuando los falsos positivos tienen un costo elevado — por ejemplo, si un sistema bloquea automáticamente tarjetas de crédito ante una sospecha de fraude, demasiados falsos positivos pueden provocar que clientes legítimos tengan operaciones rechazadas.

7.7 — Recall

El recall, también denominado sensibilidad, responde a otra pregunta:

> *De todos los casos positivos que realmente existían, ¿cuántos consiguió detectar el modelo?*

```
Recall = TP / (TP + FN)
Recall = 18 / (18 + 5) ≈ 0.783   →   78,3 %
```

El modelo detectó aproximadamente el 78 % de los fraudes existentes; el resto fueron falsos negativos. Aquí aparece una diferencia fundamental: la **precision** se concentra en la calidad de los positivos que el modelo encontró, y el **recall** se concentra en la cantidad de positivos reales que consiguió encontrar.

7.8 — Precision y recall representan problemas diferentes

Un modelo muy conservador, que solamente marca como positivo aquellos casos de los que está prácticamente seguro, probablemente tendrá una precision elevada — pero puede dejar pasar muchos casos, resultando en un recall menor. Por el contrario, un modelo mucho más sensible puede detectar casi todos los casos reales, aumentando el recall, pero al costo de generar muchos falsos positivos.

> *No existe una regla universal que diga "siempre debemos maximizar precision" o "siempre debemos maximizar recall". La métrica adecuada depende del problema y del costo de cada tipo de error.*

7.9 — F1-score

Cuando necesitamos considerar conjuntamente precision y recall podemos utilizar el F1-score, definido como la media armónica entre ambas:

```
F1 = 2 · (Precision · Recall) / (Precision + Recall)
```

La media armónica hace que un valor muy bajo de precision o recall penalice considerablemente el resultado final: un modelo no puede obtener un F1 elevado simplemente teniendo una de las dos métricas muy alta

mientras la otra es muy baja. Resulta especialmente útil cuando necesitamos un equilibrio entre detectar correctamente los positivos y evitar clasificar incorrectamente demasiados negativos como positivos.

7.10 — ¿Cuál métrica debemos utilizar?

No existe una respuesta universal. En un sistema donde queremos detectar la mayor cantidad posible de casos peligrosos, puede ser especialmente importante minimizar los falsos negativos, y el recall puede tener prioridad. En un sistema donde una falsa alarma genera un costo elevado, necesitamos prestar especial atención a la precision. En situaciones donde hay que equilibrar ambos objetivos, F1 puede ser una alternativa adecuada. Accuracy puede resultar perfectamente válida cuando las clases están relativamente equilibradas y los diferentes errores tienen una importancia similar.

> ***La elección de la métrica debe comenzar por una pregunta mucho más general que memorizar cualquier fórmula: ¿qué significa equivocarse en este problema?***

Parte II — Evaluación de regresión

La regresión necesita otro conjunto de métricas porque la salida del modelo es numérica, no una clase.

7.11 — MAE (Mean Absolute Error)

Una de las medidas más intuitivas es el MAE: calcula la diferencia absoluta promedio entre los valores reales y las predicciones.

```
MAE = (1/n) · Σ |y_i − ŷ_i|

Errores absolutos: 10, 20, 30, 40
MAE = (10 + 20 + 30 + 40) / 4 = 25
```

Podemos interpretarlo directamente en las unidades de la variable objetivo: si estamos prediciendo precios en pesos, un MAE de 25.000 significa que, en promedio, las predicciones se alejan aproximadamente $25.000 del valor real.

7.12 — MSE y RMSE

El MSE (Mean Squared Error) eleva los errores al cuadrado antes de promediarlos:

```
MSE = (1/n) · Σ (y_i − ŷ_i)²
```

Esto tiene una consecuencia importante: los errores grandes reciben una penalización mucho mayor. Si un error es 2, su contribución al cuadrado es 4; si el error es 10, su contribución es 100. Por eso MSE es especialmente sensible a errores grandes. El **RMSE** (Root Mean Squared Error) consiste en obtener la raíz cuadrada del MSE, lo que permite volver a las unidades originales de la variable objetivo.

> *MAE y RMSE pueden contar historias diferentes sobre un modelo, especialmente cuando existen algunos errores particularmente grandes.*

7.13 — R^2 (coeficiente de determinación)

Su interpretación requiere algo más de cuidado que simplemente decir que "cuanto más alto mejor". De manera general, R^2 permite evaluar qué tan bien el modelo explica la variabilidad observada en los datos en relación con un modelo de referencia basado en la media. Un valor cercano a **1** indica un ajuste muy elevado; un valor cercano a **0** indica que el modelo no mejora demasiado respecto de esa referencia. Incluso pueden aparecer valores **negativos** cuando el modelo tiene un desempeño peor que la predicción basada simplemente en la media. Por eso tampoco debemos interpretar R^2 de manera aislada.

Parte III — Train, validation, test y validación cruzada

7.14 — El problema fundamental: ¿con qué datos evaluamos?

Tenemos 5.000 registros en nuestro dataset. Podríamos utilizar todos para entrenar el modelo y después calcular su accuracy utilizando esos mismos registros. Pero estaríamos haciendo algo problemático: el modelo acaba de aprender utilizando esos datos y luego le estamos preguntando qué tan bien reproduce esos mismos datos.

> *Sería parecido a entregar a un estudiante exactamente las preguntas del examen antes de rendir y después evaluar su rendimiento con esas mismas preguntas. El resultado no nos permitiría saber si realmente aprendió. Necesitamos separar los datos.*

7.15 — Train, validation y test

Una división conceptual habitual separa los datos en tres conjuntos, cada uno con una función distinta:

- **Train:** se utiliza para entrenar el modelo.
- **Validation:** se utiliza para tomar decisiones durante el desarrollo, comparar configuraciones y ajustar hiperparámetros.
- **Test:** se reserva para la evaluación final, una instancia que el modelo no utilizó para tomar decisiones durante la construcción.

EJEMPLO SOBRE 5.000 REGISTROS

Train — 3.500	Validation — 750	Test — 750

Los porcentajes concretos pueden variar según el problema y el tamaño del dataset. Lo importante no es memorizar una división exacta, sino comprender la función de cada conjunto.

7.16 — ¿Por qué no debemos usar el test durante el ajuste?

Supongamos que entrenamos cinco modelos y evaluamos todos utilizando el conjunto de test. Observamos que el modelo número 4 obtiene el mejor resultado y entonces modificamos algunos hiperparámetros para intentar mejorarlo. Volvemos a mirar el test. Probamos otra configuración y volvemos a mirar el test otra vez.

> *Aunque no estemos modificando directamente los datos del test, estamos utilizando indirectamente esa información para tomar decisiones. El test deja entonces de ser una evaluación verdaderamente independiente. Durante el desarrollo trabajamos con train y validation; recién cuando seleccionamos nuestro modelo final, evaluamos sobre test.*

7.17 — Validación cruzada (K-Fold)

Cuando tenemos un dataset limitado, una única partición entre entrenamiento y validación puede depender demasiado de qué observaciones quedaron en cada conjunto. En **K-Fold Cross Validation** dividimos los datos en K partes. En una primera iteración utilizamos una partición para validar y las otras para entrenar; después cambiamos la partición utilizada para validar, y repetimos el proceso hasta que cada fold haya sido utilizado como validación. Finalmente calculamos el promedio de los resultados obtenidos.

K-FOLD CROSS VALIDATION CON K = 5

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Iter. 1	Val	Train	Train	Train	Train
Iter. 2	Train	Val	Train	Train	Train
Iter. 3	Train	Train	Val	Train	Train
Iter. 4	Train	Train	Train	Val	Train
Iter. 5	Train	Train	Train	Train	Val

La ventaja es que cada observación tiene la posibilidad de formar parte del conjunto de validación durante alguna iteración. Esto proporciona una evaluación más robusta que depender exclusivamente de una única partición.

7.18 — Limitaciones de la validación cruzada

La validación cruzada no es una solución mágica. Su principal costo es computacional: si realizamos cinco entrenamientos para evaluar una configuración, necesitamos entrenar el modelo varias veces. Además, no reemplaza automáticamente al conjunto de test cuando queremos realizar una evaluación final independiente.

> *También debemos tener cuidado con problemas donde existe dependencia temporal o estructura particular en los datos: no siempre podemos dividir aleatoriamente las observaciones sin considerar la naturaleza del problema. Las técnicas de Machine Learning deben aplicarse teniendo en cuenta el contexto de los datos.*

Parte IV — Generalización, underfitting y overfitting

7.19 — Generalización

Llegamos a uno de los conceptos centrales de la clase. Un modelo generaliza cuando puede utilizar lo aprendido a partir de los datos de entrenamiento para realizar predicciones razonables sobre datos nuevos.

Supongamos que entrenamos un modelo con información de clientes de enero a octubre y posteriormente lo utilizamos con clientes de noviembre. Si mantiene un buen desempeño, podemos decir que está generalizando razonablemente bien.

> *El verdadero objetivo del aprendizaje no es memorizar enero a octubre. El objetivo es aprender patrones que continúen siendo útiles cuando aparezcan observaciones que el modelo nunca había visto.*

7.20 — Error de entrenamiento y error de validación

Podemos representar el comportamiento de un modelo comparando ambos errores. Si obtenemos Train = 5 % y Validation = 7 %, la diferencia es relativamente pequeña. Pero si obtenemos Train = 1 % y Validation = 35 %, la diferencia es enorme: el modelo funciona extraordinariamente bien sobre los datos de entrenamiento, pero su comportamiento empeora considerablemente sobre datos que no utilizó para aprender.

Escenario	Error train	Error validation	Lectura
Saludable	5 %	7 %	Diferencia pequeña
Alerta	1 %	35 %	Señal clara de sobreajuste

7.21 — Underfitting

El underfitting, o subajuste, ocurre cuando el modelo es demasiado simple para capturar adecuadamente la estructura existente en los datos: no logra aprender suficientemente bien ni siquiera los datos de entrenamiento.

Imaginemos que intentamos representar una relación claramente curva utilizando una línea recta demasiado restrictiva: el modelo simplemente no tiene suficiente capacidad para representar la relación.

> *El problema no necesariamente consiste en que necesitemos "más datos". Puede estar relacionado con la representación de las características, la elección del modelo, sus hiperparámetros o su nivel de complejidad.*

7.22 — Overfitting

El overfitting, o sobreajuste, aparece cuando el modelo se adapta excesivamente a los datos de entrenamiento. Una situación típica sería Train = 99 % y Validation = 72 %: el modelo parece excelente durante el entrenamiento, pero pierde desempeño sobre datos nuevos.

> *Una forma intuitiva de entenderlo es pensar que el modelo no solamente aprendió patrones generales, sino también detalles particulares del conjunto de entrenamiento, incluyendo posiblemente ruido. Esto no significa literalmente que el modelo "memorice" todos los datos, pero la idea de memorización resulta útil como intuición inicial. El problema fundamental es la falta de generalización.*

7.23 — Complejidad y generalización

Podemos imaginar la complejidad del modelo sobre un eje horizontal, entre dos extremos problemáticos:

Baja complejidad → Underfitting	Zona de equilibrio	Alta complejidad → Overfitting

> ***No buscamos el modelo más simple posible. Tampoco buscamos el modelo más complejo posible. Buscamos una configuración que permita aprender los patrones relevantes y generalizar adecuadamente.***

7.24 — Sesgo y varianza (bias-variance trade-off)

El **bias** representa el error asociado a utilizar un modelo con supuestos demasiado restrictivos: un modelo excesivamente simple puede presentar un sesgo elevado porque no tiene capacidad suficiente para representar la relación real. La **variance**, por otra parte, está relacionada con la sensibilidad del modelo respecto de los datos de entrenamiento: un modelo demasiado complejo puede cambiar significativamente cuando cambia el conjunto de entrenamiento.

Mucho bias	Mucha variance
— Modelo demasiado simple	— Modelo demasiado sensible
— No captura la relación real	— Cambia mucho entre entrenamientos
— Conduce a underfitting	— Conduce a overfitting

El objetivo es encontrar un equilibrio entre ambos extremos.

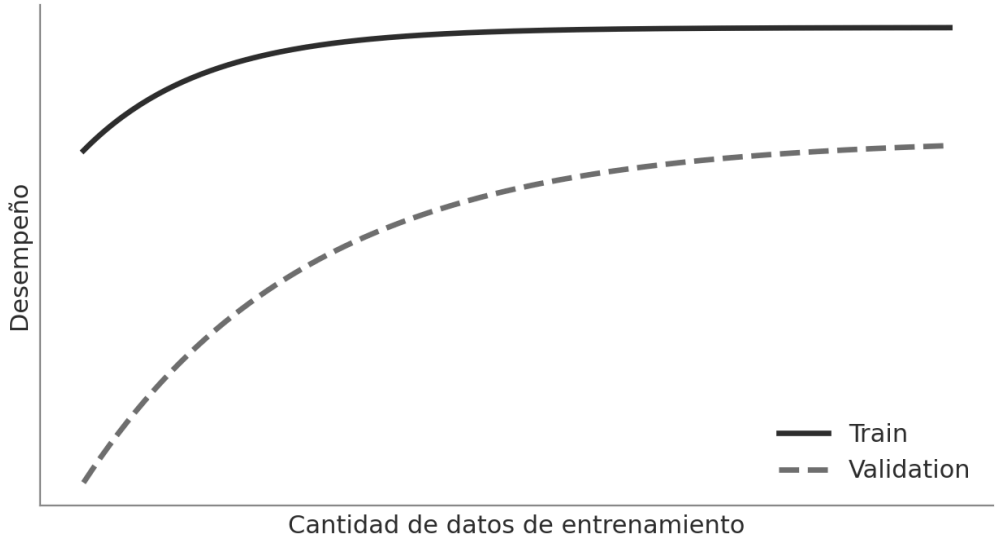
7.25 — Error irreducible

Incluso construyendo un modelo muy sofisticado, no necesariamente podremos eliminar todo el error. Los datos reales contienen ruido, información incompleta, mediciones imperfectas y factores que no están disponibles para el modelo. A esa componente que no podemos eliminar simplemente aumentando la complejidad del algoritmo se la denomina error irreducible.

> *Esto evita una expectativa equivocada: no existe un modelo perfecto simplemente porque dispongamos de suficiente capacidad computacional. El objetivo es construir modelos suficientemente buenos para el problema real y suficientemente robustos para generalizar.*

7.26 — Curvas de aprendizaje

Una herramienta útil para analizar el comportamiento de un modelo son las curvas de aprendizaje: estudian cómo evolucionan determinadas métricas cuando aumenta la cantidad de datos utilizados para entrenar.



Esquema conceptual: a medida que aumentan los datos, train y validation tienden a acercarse.

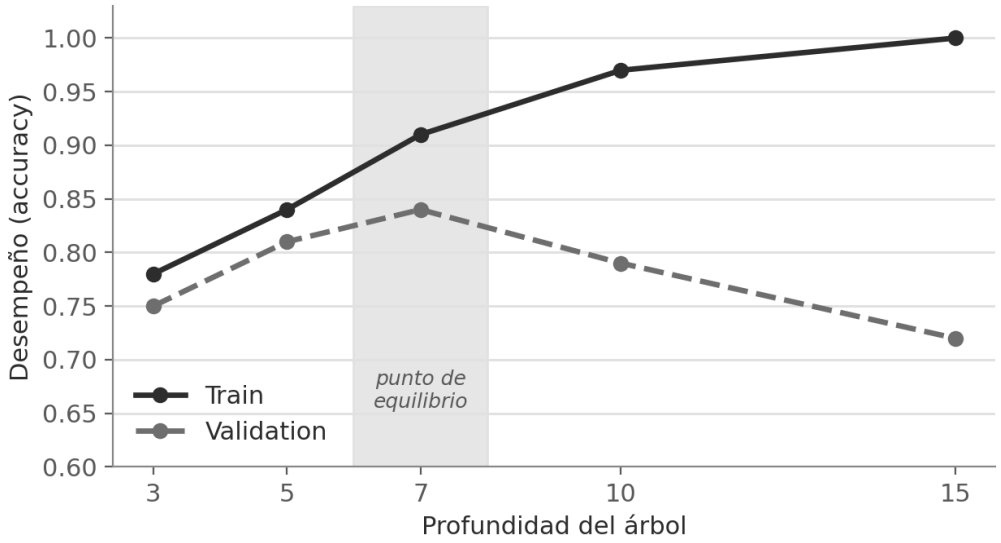
Si el modelo obtiene un desempeño excelente en entrenamiento pero existe una diferencia importante y persistente respecto de validation, podemos sospechar sobreajuste. Si ambos desempeños son bajos y permanecen bajos, podemos sospechar que el modelo no tiene suficiente capacidad, o que existe otro problema en la representación de los datos. Las curvas no sustituyen al análisis, pero ayudan a visualizarlo.

7.27 — Curvas de validación

También podemos analizar el comportamiento del modelo frente a diferentes valores de un hiperparámetro. Supongamos que entrenamos un árbol de decisión con diferentes profundidades:

Profundidad	Train	Validation
3	0,78	0,75
5	0,84	0,81
7	0,91	0,84
10	0,97	0,79

Profundidad	Train	Validation
15	1,00	0,72



La profundidad 15 logra un entrenamiento perfecto, pero validation empeora considerablemente.

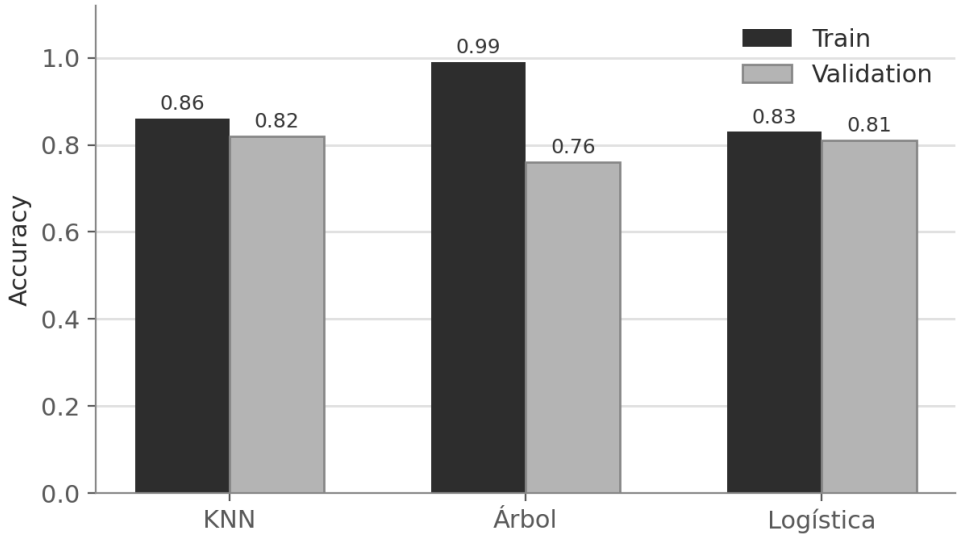
En cambio, una profundidad intermedia presenta un mejor equilibrio. Esto muestra de manera experimental que aumentar la complejidad no garantiza un mejor modelo.

Parte V — Aplicación práctica y cierre

7.28 — Evaluar los modelos de la clase anterior

Podemos volver ahora a los modelos que estudiamos anteriormente. Supongamos que entrenamos KNN, un árbol de decisión y regresión logística para predecir compra / no compra:

Modelo	Train	Validation
KNN	0,86	0,82
Árbol	0,99	0,76
Logística	0,83	0,81



El árbol muestra la mayor diferencia entre train y validation: una señal de posible overfitting.

Si observamos solamente train, podríamos elegir inmediatamente el árbol. Pero si observamos validation, la situación cambia: el árbol presenta una diferencia considerable entre entrenamiento y validación, lo que nos da una señal de posible overfitting. KNN y regresión logística presentan comportamientos más cercanos entre entrenamiento y validación.

Pero todavía falta una parte fundamental: ¿qué tipos de errores están cometiendo? Podría ocurrir que KNN tenga una accuracy ligeramente inferior pero un recall considerablemente mayor. En ese caso, dependiendo del objetivo del sistema, KNN podría ser preferible.

> ***No debemos preguntar simplemente "¿cuál tiene mayor accuracy?". Debemos preguntar: ¿cuál resuelve mejor nuestro problema y generaliza adecuadamente?***

7.29 — El flujo de evaluación

A esta altura podemos integrar todo lo aprendido en un flujo de trabajo que se reutilizará prácticamente durante toda la materia:

FLUJO DE EVALUACIÓN DE UN MODELO

Datos	**Train / Val / Test**	**Entrenamiento**
Predicción	**Métricas**	**Análisis de errores**
Ajuste de hiperparámetros	**Nuevo experimento**	**Selección del modelo**
Evaluación final con Test		

7.30 — Un modelo "bueno" depende del problema

Supongamos que tenemos dos modelos:

Modelo	Accuracy	Recall
Modelo A	95 %	60 %
Modelo B	92 %	90 %

¿Cuál es mejor? No podemos responder sin conocer el problema. Si estamos intentando detectar fraudes y perder un fraude resulta muy costoso, probablemente el modelo B sea más interesante. Si el sistema genera acciones automáticas muy costosas ante cada falso positivo, quizás necesitemos prestar mayor atención a precision.

> ***La evaluación no consiste en encontrar el número más grande. Consiste en relacionar las métricas con el objetivo real del sistema. Esta idea es fundamental para evitar uno de los errores más frecuentes al comenzar con Machine Learning: optimizar una métrica sin preguntarse si esa métrica representa realmente lo que importa.***

7.31 — Cierre

Un modelo debe ser evaluado sobre datos que permitan determinar si realmente aprendió patrones útiles. Para clasificación vimos que la matriz de confusión permite distinguir verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, y a partir de ellos construimos accuracy, precision, recall y F1-score, comprendiendo que cada métrica responde una pregunta diferente. Para regresión introdujimos MAE, MSE, RMSE y R^2, observando que también aquí la elección de la métrica depende de aquello que queremos medir.

Después analizamos la separación entre train, validation y test, y comprendimos por qué no debemos utilizar el conjunto de prueba para ajustar continuamente nuestro modelo. Introdujimos también la validación cruzada como una estrategia para obtener evaluaciones más robustas cuando disponemos de una cantidad limitada de datos.

Finalmente llegamos al concepto que conecta toda la clase: **generalización**. Un modelo no es bueno porque memoriza los datos de entrenamiento; es bueno cuando logra utilizar lo aprendido para realizar predicciones razonables sobre datos que nunca vio. El underfitting aparece cuando el modelo resulta demasiado simple y no consigue aprender adecuadamente. El overfitting aparece cuando el modelo se adapta excesivamente al entrenamiento y pierde capacidad de generalización. El compromiso entre bias y variance nos ayuda a comprender por qué existe una zona intermedia en la que debemos buscar el equilibrio adecuado entre capacidad y generalización.